

Whitebox Kriptografi

CEN429 Güvenli Programlama — Hafta 11

Dr. Öğr. Üyesi Uğur CORUH · 27.11.2026

Bugünün planı (3 saat)

Zaman	Bölüm	Ne yapılıyor
0:00–0:25	1	Kara/gri/beyaz kutu modelleri; WBC saldırgan yetenekleri
0:25–0:50	2	Problem neden zor; naif çözümler: gömülü anahtar, XOR, code lifting
0:55–1:40	3	Tablo tabanlı WBC (Chow AES): kodlama, bijeksiyonlar, tablo boyutu/hız
1:45–2:20	4	Saldırı geçmişi: BGE, DFA, DCA, WhibOx; "hepsi kırıldı" ve karşı önlemler
2:20–2:45	5	Katmanlı savunmadaki yeri; donanım alternatifleri (TEE/SE, HSM/SoftHSM)
2:45–3:00	6–7	Hatalı→saldırı→koruma; proje S8; kendini sına

Bu hafta nereye oturuyor?

- **9. hafta:** kod gizleme — "gizleme anahtarı korumaz" dedik
- **Bu hafta (11):** peki anahtarı yazılımda **nasıl** koruruz? → whitebox
- **10. hafta:** kriptonun doğru kullanımı, anahtar yaşam döngüsü

Bugün "anahtar saldırganın elindeyken ne yaparız?" sorusunu işliyoruz.

Öğrenme çıktısı

Bu hafta **ÖÇ.2 / ÖÇ.4** (kriptografi, güvenli iletişim) üstünedir.

Sonunda yapabileceğiniz:

- Kara/gri/beyaz kutu modellerini ayırmak
- Whitebox'ın **ne yaptığını ve neden yetmediğini** anlatmak
- Bir anahtar için doğru koruma kararı vermek

Baştan bir uyarı

Bugüne kadar yayımlanmış **saf yazılım** whitebox tasarımlarının **hepsi kırıldı**.

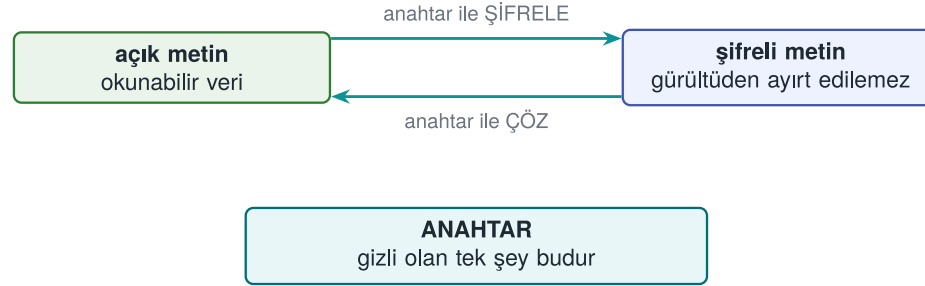
O yüzden bugün "whitebox = güvenli anahtar" demeyeceğiz.

Diyeceğiz ki: whitebox, anahtar çıkarmayı **geciktiren bir katmandır** — ve tek başına yeterli değildir.

Önceki haftalardan gelenler (1)

Şifreleme nedir? İki yönlü, anahtara bağlı dönüşüm

aynı anahtar hem kilitler hem açar (simetrik şifreleme)



Kerckhoffs ilkesi: algoritma herkesçe bilinebilir; güvenlik yalnız anahtarın gizliliğindedir.

- **Şifreleme, anahtar, simetrik/asimetrik** — açık metni bir anahtarla okunamaz hale getirmek; güvenlik anahtarın gizliliğine dayanır, algoritmanın gizliliğine değil (**Hafta 3**)
- **AES ve bit güvenlik düzeyi** — en yaygın simetrik algoritma, 16 baytlık bloklar, turlar; "n-bit güvenlik" $\approx 2^n$ deneme (**Hafta 10**)
- **XOR** — bitler farklıysa 1, aynıysa 0; $a \oplus b = a$ ile geri alınabilir; AES'te anahtar ekleme adımı budur (**Hafta 5**)
- **Entropi** — bir verinin baytlarının ne kadar rastgele görüldüğünün ölçüsü; şifreli/rastgele bloklar yüksek entropili görünür (**Hafta 2**)

Bu hafta bu araçları **whitebox** bağlamında yeniden kullanıyoruz.

Önceki haftalardan gelenler (2)

- **Hata ayıklayıcı (debugger)** — bir programı adım adım çalıştırıp bellek/kayıtları okuyan araç (gdb, x64dbg) (Hafta 6)
- **Statik ve dinamik analiz** — çalıştırmadan / çalıştırarak yapılan inceleme; saldırıları sınıflamakta kullanacağız (Hafta 4)
- **Enstrümantasyon ve emülatör** — bir programı otomatik izleyen ve gerçek işlemciyi yazılımla taklit eden araçlar (Hafta 6)
- **HSM ve SoftHSM** — anahtarı hiç dışarı çıkarmadan işleyen özel donanım ve onun yazılım benzetimi (**Hafta 10**)

Bu hafta bunları whitebox saldırganının araç kutusu ve savunma seçenekleri olarak yeniden görüyoruz.

Bu haftanın kavramları

Her terim, gövdede ilk geçtiği yerde tanımlanır; burada yalnız **nerede** olduğunu işaretliyoruz.

Kavram	Nerede
Kara/gri/beyaz kutu, WBC saldırgan modeli	Bölüm 1
Problem neden zor; naif çözümler (gömülü anahtar, XOR, code lifting)	Bölüm 2
Tablo tabanlı WBC: kısmi değerlendirme, iç/dış kodlama, bijeksiyon	Bölüm 3
Saldırı geçmiş: BGE, DFA, DCA, WhibOx	Bölüm 4
Katmanlı savunmadaki yer; TEE/SE, HSM/SoftHSM	Bölüm 5
Hatalı → saldırı → koruma senaryosu	Bölüm 6
Proje S8, kendini sına	Bölüm 7

Ön bilgi · S-box nedir?

- **S-box (substitution box):** bir baytı başka bir baytla değiştiren **sabit bir tablo**.
- AES'in "bayt değiştirme" adımı budur.
- Örnek: `S-box[0x53] = 0xED` gibi 256 girişli bir eşleme.

Bu tabloyu aklınızda tutun; whitebox'ın kalbi budur.

Arama tablosu (lookup table)

- **Arama tablosu:** "girdiye karşılık çıktı" veren dizi.
- $T[x]$ = x için önceden hesaplanmış sonuç.
- Hesap yapmak yerine **tablodan okursunuz**; hızlıdır.

Whitebox, hesabı tablolara gömer.

Ön bilgi · Yan kanal (side channel) nedir?

- **Yan kanal:** algoritmanın matematiğini değil, çalışırken **sızdırdığı fiziksel/dolaylı bilgiyi** kullanan saldırı.
- Örnek: harcanan **zaman**, **güç** tüketimi, elektromanyetik yayım.

Kartlarda klasiktir; birazdan whitebox'ta yazılım karşılığını göreceğiz.

DPA nedir?

- **DPA (Differential Power Analysis):** bir cihazın **güç tüketimi** ölçümlerine istatistik uygulayıp anahtarı çıkaran yan kanal saldırısı.
- Cihazın içine bakmaz; dışarıdan ölçer.

Bu fikri aklınızda tutun; whitebox'ın en güçlü saldırısı (DCA) bunun yazılım hâlidir.

Ön bilgi · Bijeksiyon (birebir-örten eşleme)

- **Bijeksiyon:** her girdiyi tam bir çıktıya eşleyen, **tersi alınabilen** dönüşüm.
- Örnek: $f(x) = x \wedge 0x5A$ bir bijeksiyondur (tersi yine kendisi).

Whitebox, tabloları **gizli bijeksiyonlarla** sarar.

Ön bilgi · TEE ve güvenli öge (SE)

- **TEE (Trusted Execution Environment):** telefon işlemcisinin, işletim sisteminden **yalıtılmış** güvenli bölgesi.
- **Güvenli öge (SE):** anahtarları saklayan ayrı, kurcalamaya dirençli **donanım** çip.

Anahtar için en güçlü koruma bunlardır; whitebox bunların **olmadığı** durum içindir.

Şimdi hazırız

Bildiğimiz terimler:

şifreleme/çözme · anahtar · simetrik/asimetrik · AES · S-box · arama tablosu · XOR · bit güvenlik · yan kanal · DPA · bijeksiyon · TEE/SE · HSM/SoftHSM

Şimdi asıl soruya: **anahtar saldırganın elindeyken ne yaparız?**

1. Üç saldırgan modeli

Kısa tarihçe — whitebox kriptografi

- **1883** — **Kerckhoffs**: güvenlik **anahtarda** olmalı, sistemin gizliliğinde değil
- **2002** — Chow vd. ilk **whitebox AES/DES** (DRM için) — WBC'nin doğuşu
- **2004** — **BGE saldırısı** ilk WB-AES'i kırar
- **2016** — **DCA** (Bos vd.): donanım DPA'sı yazılıma taşınır, otomatik kırar
- **2017–2024** — **WhibOx**: yayımlanan tüm saf-yazılım adaylar kırıldı

Bugünkü kural: WBC bir **geciktirme katmanı**; mümkünse **donanım** (TEE/SE/HSM).

Neden "model"?

Bir savunmayı tasarlamadan önce **saldırganın neyi görebildiğini** belirlemeliyiz.

Buna **saldırgan modeli** denir.

Üç model var: kara, gri, beyaz kutu.

Kara kutu

- Saldırgan **yalnız girdi ve çıktıyı** görür.
- İçeriği (anahtar, ara değerler) göremez.
- Tipik ortam: **uzak sunucu** (kod sizde, saldırgan ağdan bağlanıyor).

AES/RSA'nın güvenlik kanıtları bunu varsayar.

Gri kutu

- Saldırgan girdi/çıktıya ek olarak **yan kanalları** görür (zaman, güç, EM).
- İçeriği doğrudan okuyamaz ama **dolaylı** ölçer.
- Tipik ortam: elinde tuttuğu bir **akıllı kart** ya da gömülü cihaz.

Beyaz kutu

- Saldırgan **her şeyi** görür: kod, bellek, tüm ara değerler.
- Üstelik **değiştirebilir**.
- Tipik ortam: **yazılım**, saldırganın kendi cihazında çalışıyor.

Bu, mobil/masaüstü uygulamanın gerçek durumudur.

Üç model bir arada

Model	Ne görür / yapar	Ortam
Kara	Girdi/çıktı	Uzak sunucu
Gri	+ yan kanal	Akıllı kart
Beyaz	Her şey + değiştirir	Yazılım, kendi cihazı

Kripto **kara kutu** için tasarlandı; biz **beyaz kutu**dayız.

Üç saldırgan modeli — ne görüyor?

Üç saldırgan modeli: ne görüyor?

soldan sağa görünürlük artar



WBC en zor modelde anahtar çıkarmayı GECİKTİRMEYE çalışır; imkânsız kılmaz.

Saldırgan modelleri (animasyon)

yetenek

uygulanan yetenekler (özet)

Public REST login endpoint -> IO

Rate-limited cloud pricing API -> IO

Contactless payment card, power-trace rig -> IO,SC

Car-key fob, radio timing capture -> IO,SC

Decompiled mobile banking APK -> IO,SC,MEM,COD,MOD

Debugger attached to a desktop app -> IO,SC,MEM,COD,MOD

Rooted phone running the client app -> IO,SC,MEM,COD,MOD

Web login form over HTTPS -> IO

Smart-card reader on the attacker's own desk -> IO,SC

Downloaded game client binary -> IO,SC,MEM,COD,MOD

DNS resolver answering only queries -> IO

Firmware dumped from a purchased device -> IO,SC,MEM,COD,MOD

12 senaryonun hepsi sınıflandırıldı. WBC, tam olarak beyaz kutu satırının MOD ve COD sütunlarını hedef alır.

Saldırgan modelleri — uç durum: hepsi beyaz kutu

yetenek

uygulanan yetenekler (özet)

Reverse-engineering lab sample #1 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #2 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #3 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #4 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #5 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #6 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #7 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #8 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #9 -> IO,SC,MEM,COD,MOD
Reverse-engineering lab sample #10 -> IO,SC,MEM,COD,MOD

10 senaryonun hepsi sınıflandırıldı. WBC, tam olarak beyaz kutu satırının MOD ve COD sütunlarını hedef alır.

Sorun burada

AES matematiksel olarak güçlü. Ama:

- Beyaz kutuda **anahtar bellekte** bir yerdedir
- Saldırgan belleği okuyabilir
- O zaman AES'in gücü ne işe yarar?

Güçlü kilit, anahtarı kapının yanına asarsanız işe yaramaz.

Whitebox kriptografi (WBC) fikri

- **WBC:** şifrelemeyi, anahtar **hiçbir zaman açıkça bellekte olmayacak** biçimde gerçekleştirmek.
- Anahtar, önceden hesaplanmış **arama tablolarının içine** gömülür.
- Bellekte "işte anahtar bu" diyebileceğiniz bir bayt dizisi **yoktur**.

WBC bir tanım değil, bir saldırgan modeli

- WBC, 2002'de Chow ve arkadaşları tarafından tanımlandı.
- Aslında bir **saldırgan modelidir**: "saldırgan her şeyi görüp değiştirebiliyorsa şifreleme nasıl yapılır?"

WBC saldırgan neler yapabilir? (1)

Gözlemler:

- O an işlenen komutlara erişir
- Algoritma akışını izler
- Kullanılan **belleği görür**

Yani programın gördüğü her ara değeri o da görür.

WBC saldırgan neler yapabilir? (2)

Değiştirir:

- Program belleğini kurcalar
- Algoritmanın **yalnız bir turunu** çalıştırır
- `if` koşullarını değiştirir
- Döngü sayaçlarını değiştirir
- **Hata enjekte eder** (bir değeri bozar)

Kaynağın uyum maddesi

Teknik kılavuz bu bölümü şu gereksinimle açar:

"Kriptografik anahtarlar, gizlilik ve bütünlüğü korumak için **güvence altına alınmalı ve/veya gizlenmelidir.**"

Ama hemen ekliyor

Kılavuz, bu saldırgan yeteneklerine karşı koruyanın **yalnız WBC olmadığını** söyler:

Uygulamayı ve varlıkları koruyan şey, **kod sağlamlaştırma (9. hafta) ve RASP (6. hafta)** yöntemleridir.

WBC ilk cümleden itibaren **tek başına** sunulmaz.

2. Problem ve naif çözümler

Problem neden bu kadar zor?

Temel gerilim:

- Program şifrelemek için anahtara **erişmek** zorunda
- Saldırgan da programın gördüğü her şeyi görüyor
- Yani anahtar bir noktada **saldırganın da görebildiği** yerde

WBC bunu "anahtarı tablolara pişirerek" aşmaya çalışır.

Bu gizlilik değil, maliyet

WBC gizliliği kriptodan gizlemeye **taşımaz**.

Matematiksel bir dönüşümle anahtarı tablolara **dağıtır**.

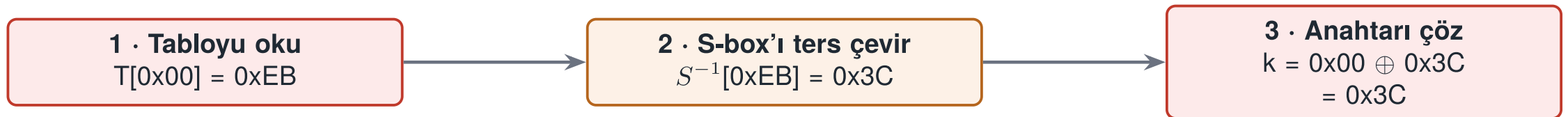
Amaç yine 9. haftadaki gibi: anahtarı geri çıkarmayı **çok daha pahalı** yapmak.

Naif çözümler neden çöker?

Anahtar tablodan nasıl sızıyor — şema

Naif tablo anahtarı nasıl sızdırır?

$T[x] = S\text{-box}[x \oplus k]$ — tek girdi yeter, 256 deneme bile gerekmez



“Anahtar kodda görünmüyor” demek, “anahtar korunuyor” demek DEĞİLDİR.

Neden çökenleri öğreniyoruz?

WBC'ye neden ihtiyaç olduğunu görmek için.

Her başarısız "kolay" çözüm, bir güvenli programlama **kuralına** dönüşür.

Naif 1 · Anahtarı diziye göm

```
static const uint8_t k[16] = { 0x2B, 0x7E, ... }; /* KÖTÜ */
```

En kolay yol, en zayıf yol.

Naif 1 · Nasıl kırılır?

1. `strings` / bayt taraması: 16 baytlık blok görünür
2. **Entropi taraması**: yüksek entropili blok "anahtar burada" der (Shamir–van Someren gözlemi)
3. Hata ayıklayıcı: şifreleme çağrısında bu bloğun kullanıldığını doğrular

Süre: dakikalar.

Naif 1 · Kural

Düz gömülü anahtar bir anahtar koruması **değildir**.

Bu, 9. haftadaki "dize gizleme anahtar saklamaz" kuralının ta kendisi.

Gömülü anahtarı tarayarak bulma (animasyon)

ikili dosya (bayt) =

07	18	29	3A	C3	9A	71	2E	55	F0	18	BD	4C	A6	37	E9	02	8F	D1	64	5B	6C	7D	8E
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

0x1000100110021003100410051006100710081009100A100B100C100D100E100F10101011101210131014101510161017

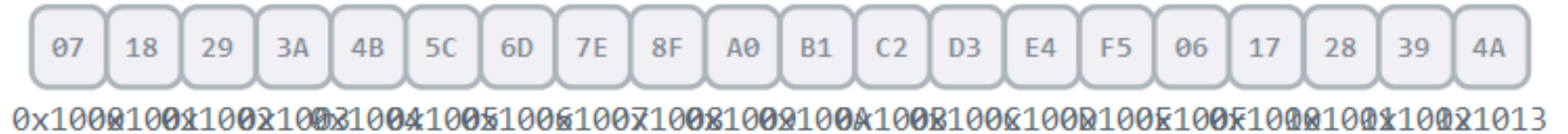
aranan KEY[16] =

C3	9A	71	2E	55	F0	18	BD	4C	A6	37	E9	02	8F	D1	64
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

SONUÇ: LEAKED — gömülü anahtar, kendi ikili dosyasında bulundu. Beyaz kutu saldırgan aynı taramayı çalıştırır.

Gömülü anahtar taraması — uç durum: anahtar yok

ikili dosya (bayt) =



aranan KEY[16] =



Gerçek demoda bu asla olmaz: anahtar KENDİ derlenmiş ikilisinin içinde durur, tarama onu her zaman bulur.

Naif 2 · XOR ile "karıştır"

```
static const uint8_t k_gizli[16] = ...;    /* k ^ maske */  
/* çalışırken: k = k_gizli ^ maske */
```

"Anahtarı bir maskeyle XOR'layıp saklayayım."

Naif 2 · Neden çöker?

- **Maske de** binary dosyadadır
- Program `k` 'yı çözebiliyorsa, saldırgan da çözer
- Sadece bir adım fazladan; gerçek bir engel değil

Program neyi yapabiliyorsa, beyaz kutu saldırgan da yapar.

Naif 3 tehdidi · Kod sökme (code lifting)

- Diyelim WBC anahtarı tablolara gömdü.
- Saldırgan anahtarı **hiç çıkarmadan**, şifreleme yapan kod parçasını (tablolar + yorumlayıcı) **olduğu gibi kopyalar**.
- Kendi programında kullanır: anahtarı bilmeden işlevini **çalar**.

Code lifting — şema

Code lifting: anahtarı hiç çıkarmadan

WBC'nin en sinsi saldırısı: anahtar gizli kalsa bile işlev çalınır

Saldırı

whitebox rutinini/tablosunu
olduğu gibi KOPYALA

kendi programında kullan
anahtar hiç bilinmez

Önlem

dış kodlama (F, G)
cihaz bağlama
sunucu tarafı denetim
taşınan rutin işe yaramaz

Bu yüzden WBC tek başına değil, bağlama ve sunucu denetimiyle birlikte kullanılır.

Kod sökme · Kural

WBC'nin "anahtarı gizledim" demesi yetmez.

Kod sökmeye (code lifting) karşı **cihaz/sürüm bağlama** gerekir (birazdan 3. bölüm): tablolar yalnız belirli bir cihazda/sürümde anlamlı olsun.

Naif çözümler: özet

Naif çözüm	Kıran
Diziye gömülü anahtar	<code>strings</code> + entropi + gdb
XOR/whitening	maske de içeride
(WBC bile)	kod sökme → cihaz bağlama gerekir

Bölüm 1–2 — kısa sinama

1. Kara, gri, beyaz kutu farkı?
2. AES güçlüyken beyaz kutuda sorun ne?
3. Anahtarı diziye gömmek neden koruma değil?

Bölüm 1–2 — cevaplar

1. **Kara:** yalnız girdi/çıktı · **Gri:** + yan kanallar (zaman/güç) · **Beyaz:** her şey + değiştirir.
2. Anahtar bir noktada **bellekte**; beyaz kutu saldırgan onu görür. AES'in gücü **kara kutu** varsayımına dayanır.
3. `strings` + **entropi** taraması + gdb ile dakikalar içinde bulunur; program onu kullanmak zorundadır.

3. Tablo tabanlı WBC (adım adım)

Ne öğreneceğiz?

Chow ve arkadaşlarının 2002 AES tasarımı, tablo tabanlı WBC'nin klasığıdır.

- **Fikri** (nasıl çalışır)
- **Maliyeti** (tablo boyutu, hız)
- **Sınırı** (neden yeterli değil)

Sınavda matematik yok; **fikir ve sınır** var.

Hatırlatma: bir AES turunun iki adımı

1. Durum baytını tur anahtarıyla **XOR**'la: $x \oplus k$
2. Sonucu **S-box**'tan geçir: $S\text{-box}[x \oplus k]$

Bu iki adım, sabit bir k için birleştirilebilir mi? Evet.

AES turu sıfırdan (animasyon)

durum (girdi, 4x4, sütun sütun dolduruldu)

	c0	c1	c2	c3
r0	AB	56	52	BF
r1	A2	65	62	70
r2	01	66	0F	94
r3	5E	A7	FE	EE

tur anahtarı (4x4)

	c0	c1	c2	c3
r0	2B	28	AB	09
r1	7E	AE	F7	CF
r2	15	D2	15	4F
r3	16	A6	88	3C

Tablo tabanlı whitebox, tam olarak BU yapıyı arama tablolarına gömer — bir sonraki animasyon.

AES turu — uç durum: durum ve anahtar sıfır

durum (girdi, 4x4, sütun sütun dolduruldu)

	c0	c1	c2	c3
r0	00	00	00	00
r1	00	00	00	00
r2	00	00	00	00
r3	00	00	00	00

tur anahtarı (4x4)

	c0	c1	c2	c3
r0	00	00	00	00
r1	00	00	00	00
r2	00	00	00	00
r3	00	00	00	00

Tablo tabanlı whitebox, tam olarak BU yapıyı arama tablolarına gömer — bir sonraki animasyon.

Adım 1 · Anahtarı tabloya "pişir"

```
Normal:  çıktı = S-box[ x ^ k ]  
WBC:    T[x]  = S-box[ x ^ k ]   (256 girişli tablo)
```

Artık kodda **k** diye bir değişken **yok**. **k**, tablonun **içine** gömülü.

Adım 1 · Ne kazandık?

- Bellekte `k` adında bir bayt dizisi yok
- Sadece `T` tablosu var (256 giriş)

Görünüşte anahtar kayboldu. Ama...

Adım 1 · Neden tek başına güvensiz?

- T , S-box'ın x^k ile **ötelenmiş** hali
- Saldırgan T 'yi bilinen S-box ile karşılaştırır
- Öteleme miktarından k 'yı **geri çıkarır**

Bu yüzden tabloları **kodlamak** gerekir.

Neden oyuncak örnek?

Gerçek AES 256 girişli tablolarla çalışır; tahtaya sığmaz.

Fikri **4 değerli** minik bir örnekle göreceğiz.

Amaç: "anahtarı tabloya pişirmek" ne demek, **elle** görmek.

Minik S-box tanımı

Diyelim 2 bitlik değerlerimiz var (0,1,2,3) ve şu sabit S-box:

x:	0	1	2	3
S-box[x]:	3	2	0	1

Bu tablo **herkese açık** (algoritmanın parçası, sır değil).

Anahtar ve işlem

- Anahtar $k = 1$ (gizli).
- İşlem (bir AES turunun mini hâli): $\text{ç1kt1} = \text{S-box}[x \wedge k]$.

$\wedge$ = XOR. $x \wedge 1 : 0 \leftrightarrow 1, 2 \leftrightarrow 3$ (son biti çevirir).

Adım adım: normal işlem

```
x=0 → x^1=1 → S-box[1]=2  
x=1 → x^1=0 → S-box[0]=3  
x=2 → x^1=3 → S-box[3]=1  
x=3 → x^1=2 → S-box[2]=0
```

Burada `k=1` kodda **açıkça** kullanılıyor. Saldırgan görür.

Anahtarı tabloya "pişir"

$T[x] = S\text{-box}[x \wedge 1]$ diye **önceden** hesaplayıp tablo yapalım:

x:	0	1	2	3
T[x]:	2	3	1	0

Artık kodda **k** **yok**; sadece **T** var. Görünüşte anahtar kayboldu.

Ama anahtar sızıyor! (kodlama yoksa)

Saldırgan T 'yi bilinen $S\text{-box}$ ile karşılaştırır:

```
S-box: 3 2 0 1  
T:      2 3 1 0
```

$T[x] = S\text{-box}[x \wedge k]$ olduğunu bilerek hangi k uyuyor diye dener: $k=1$ uyar.

Sonuç: kodlanmamış tablo anahtarı ele verir. İşte bu yüzden iç/dış kodlama var.

Tabloya anahtarı gömme (animasyon)

gizli anahtar $k = 0x3C$

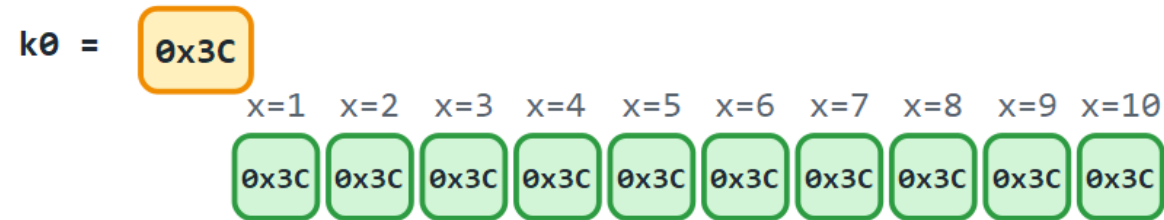
	0	1	2	3	4	5	6	7	8	9
$T[x]$	31	D8	7F	26	95	3C	E3	8A	F9	A0

	0	1	2	3	4	5	6	7	8	9
$T2[x]$	72	CF	2C	89	FE	5B	B8	15	8A	E7

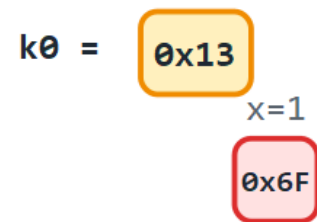
Her iki tablo da BİREBİR AYNI fonksiyonu hesaplıyor ($x \rightarrow S[x \text{ XOR } k]$); tek fark T2nin çıktısının E ile sarılmış olması. Bir sonraki animasyon (table-key-leak), bunun T2i neden anında ele verdiğini gösteriyor.

Neden tablo anahtarı ele verir? (animasyon)

[1] NAİF tablo T — $\text{Sinv}[T[x]] \text{ XOR } x$ sabit mi?



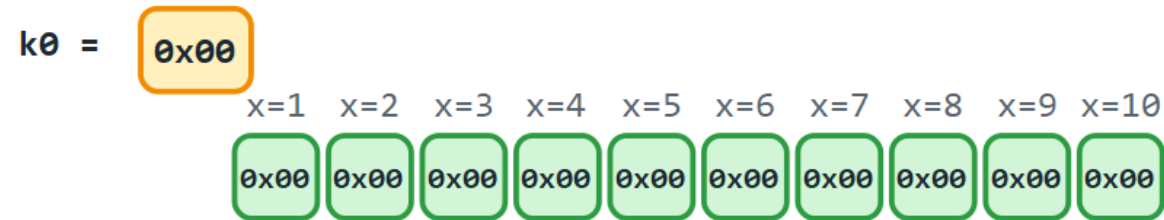
[2] KODLANMIŞ tablo T2 — $\text{Sinv}[T2[x]] \text{ XOR } x$ sabit mi?



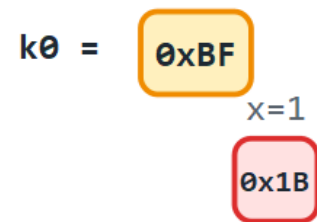
SONUÇ: Naive recovery FAILED — gizli çıkış kodlaması E, S-boxun tersini anlamsız hale getirdi; anahtar bu tablodan DOĞRUDAN okunamıyor.

Tablo sızıntısı — uç durum: sıfır anahtar

[1] NAİF tablo T — $\text{Sinv}[T[x]] \text{ XOR } x$ sabit mi?



[2] KODLANMIŞ tablo T2 — $\text{Sinv}[T2[x]] \text{ XOR } x$ sabit mi?



SONUÇ: Naive recovery FAILED — gizli çıkış kodlaması E, S-boxun tersini anlamsız hale getirdi; anahtar bu tablodan DOĞRUDAN okunamıyor.

Adım 2 · Tabloları büyüt (T-box)

- Tek bayt eşlemesi yerine, S-box ile AES'in **karıştırma** adımını (MixColumns) birleştir
- Sonuç: 8 bit girip **32 bit** çıkaran daha büyük tablolar (**T-box**)
- Baytlar arası XOR'lar da küçük **XOR tabloları** (4 bit) olur

Adım 2 · Sonuç

- Bütün tur, bir **arama tabloları ağına** dönüşür
- Hiçbir yerde açık bir aritmetik anahtar işlemi görünmez
- Ama tablolar hâlâ tek başına incelenebilir → kodlama şart

Adım 3 · İç kodlamalar (internal encodings)

- Her tablonun **çıkışına** gizli bir bijeksiyon uygula
- Bir sonraki tablonun **girişinde** onun tersini uygula
- Ardışık kodlamalar birbirini **götürür** → sonuç doğru

Ama **ara değerler karışık** görünür.

İç kodlama — şema

İç kodlama ne değiştirir?

gizli bijeksiyon G , ara değeri maskeler

Kodlamasız tablo

$$T[x] = S\text{-box}[x \oplus k]$$

S^{-1} uygulanabilir
→ k geri hesaplanır

İç kodlamalı tablo

$$T'[x] = G(S\text{-box}[x \oplus k])$$

G bilinmeden S^{-1} yok
→ adım 2 kırılır

Ardışık tabloların kodlamaları birbirini götürür: sonuç doğru, ara değer karışık.

Adım 3 · Neyi engeller?

Bir tabloyu **tek başına** inceleyen saldırganı durdurmaya amaçlar.

Çünkü o tablonun girişi/çıkışı, gizli bir kodlamayla bozulmuştur.

Kodlama fikri (minik)

Çıkışa gizli bir eşleme E uygula: $T'[x] = E(T[x])$.

$E: 0 \rightarrow 1, 1 \rightarrow 3, 2 \rightarrow 0, 3 \rightarrow 2$ (gizli bijeksiyon)
 $T': E(2) E(3) E(1) E(0) = 0 2 3 1$

Artık T' , S -box'a benzemez; basit karşılaştırma k 'yi vermez.

Kodlamanın bedeli

- Bir sonraki adım **E** 'nin **tersini** uygulamalı ki sonuç doğru çıksın.
- Gerçek AES'te bu, tabloları **zincirleyerek** yapılır (iç kodlamalar).
- Her kodlama fazladan tablo ve boyut demektir → **maliyet**.

Minik örnek bile fikri gösteriyor: gizlilik ucuz değil.

Adım 4 · Karıştırıcı bijeksiyonlar

- İç kodlamalara ek olarak, $GF(2)$ üzerinde doğrusal **karıştırıcı matrisler** (8×8 , 32×32)
- Bunlar bilgiyi tablolar arasında **yayar**
- Bir tablodan sızan bilgi tek başına anlamlı olmaz

($GF(2)$ = bitlerle, XOR aritmetiğiyle çalışan matematik; ayrıntı gerekmez.)

Adım 4 · Tablo tipleri

- Chow'un tasarımında bu kodlanmış/karıştırılmış tabloların **tipleri** vardır (literatürde I–IV).
- Her tip farklı bir rol üstlenir.
- Ders için: "tablolar tek tip değil, çeşitli" bilmek yeter.

Adım 5 · Dış kodlamalar F ve G

- En dışta, tüm şifre iki gizli bijeksiyonla sarılır: **F** ve **G**
- Kaynak kılavuz: "whitebox AES işlemi F ve G adlı iki rastgele tekil-olmayan matrisle kapsülленir"
- Yani ağ artık saf AES değil:

Dış kodlama: en güçlü koruma, en büyük sınır

çıkı artık STANDART AES değildir



Karşı taraf F ve G'yi bilmeden sonuç anlamsız → yalnız iki ucu da sizin olduğu protokolde.

Adım 5 · Dış kodlama neden güçlü?

- Saldırganın gördüğü giriş/çıkış **standart AES'in giriş/çıkışı değil**
- Tek başına tablolara bakan pek çok saldırıyı (istatistiksel/cebirsel) çok zorlaştırır

Adım 5 · Ama en büyük sınır da bu

$G \circ AES \circ F^{-1}$ standart AES değildir.

- Karşı taraf (sunucu/başka uç) F ve G 'yi bilip geri almadan sonuç **anlamsız**
- Yani dış kodlamalı WBC yalnız **iki ucu da sizde** olan protokolde kullanılır
- EMV gibi **standart AES bekleyen** bir protokolde kullanılamayabilir

Beş adım · özet

1. Anahtarı tabloya pişir
2. Tabloları büyüt (T-box + XOR tabloları)
3. İç kodlamalar
4. Karıştırıcı bijeksiyonlar
5. Dış kodlamalar (F, G)

Sonuç: anahtar açıkça yok; her şey kodlanmış tablolarda.

Minik örnekten çıkan ders

1. Anahtarı tabloya gömmek onu **görünmez** yapmaz (kodlama şart)
2. Kodlama işi çözer ama **boyut/karmaşıklık** ekler
3. Gerçekte tablolar yüzlerce KB olur

Bu, bölüm 2'deki beş adımın **sezgisidir**.

WBC'nin maliyeti

WBC bedava değil (1)

Anahtarı tablolara gömmenin bedeli: **kocaman tablolar**.

Ölçüt	Yaklaşık (mertebe)
Tablo boyutu	Yüzlerce KB (~770 KB)
Karşılaştırma	Std AES anahtarı: 16–32 bayt

WBC bedava değil (2)

Ölçüt	Yaklaşık (mertebe)
Tur başına tablo okuması	Binlerce (~3000)
Hız	Std AES'e göre ~50× yavaş

Ölçü hissi: tablo boyutu hesabı

Örnek: 288 tablo $\times$ 1 KB $\approx$ **294.912 bayt** ($\sim$ 288 KB).

- Amaç kesin sayı değil, **mertebe**
- Bu yüzden WBC yalnız **seçili, kritik** işlemlere uygulanır
- Aynı mantık 9. haftadaki sanallaştırma (K-10) ile birebir

Bölüm 3 — kısa sinama

1. $T[x] = S\text{-box}[x \wedge k]$ tek başına neden güvensiz?
2. İç kodlama ile dış kodlama farkı?
3. Dış kodlamanın (F, G) en büyük sınırı ne?
4. WBC neden yalnız seçili işlemlere uygulanır?

Bölüm 3 — cevaplar

1. T , bilinen S-box'ın x^k ile ötelenmiştir; iki tablo karşılaştırılıp **k geri çıkar**.
2. **İç**: tablolar arası ara değerleri karıştırır · **Dış (F,G)**: tüm şifreyi sarar ($G \circ \text{AES} \circ F^{-1}$).
3. Artık **standart AES değildir**; karşı taraf F/G'yi bilmeli — EMV gibi standartlarda kullanılamayabilir.
4. Tablolar yüzlerce KB + $\sim 50\times$ yavaş; maliyet yüzünden yalnız **küçük/kritik** işlemler.

4. Saldırı geçmişi ve karşı önlemler

WBC kırılma tarihi — şema

Whitebox kriptografi: kırılma tarihi



Sonuç: WBC bir GECİKTİRME katmanıdır; mümkünse donanım tercih edilir.

Neden geçmişe bakıyoruz?

Bir korumaya ne kadar güveneceğinizi, onun **kırılma geçmişi** söyler.

Her satır, savunmacı için bir **çıkarım**dır.

2002 · Doğuş

- Chow ve arkadaşları WB-AES ve WB-DES tasarımlarını yayımlar.
- Tablo tabanlı yaklaşım doğar.

Çıkarım: fikir umut verici; ama sınanmadı.

2004 · BGE saldırısı

- **BGE** (Billet–Gilbert–Ech–Chatbi): Chow AES'i pratik sürede kırar ($\sim 2^{30}$ işlem).
- İç kodlamaları çözer.

Çıkarım: iç kodlamalar tek başına **yetmez**.

2007 · WB-DES düşer

- Goubin, Wyseur ve diğerleri WB-DES ve harici kodlamalı türevlerini kırar.

Çıkarım: DES tabanlı WBC bırakılır.

2010'lar · "Daha çok tablo" denemesi

- Karroumi'nin "çift AES" güçlendirmesi de kırılır ($\sim 2^{22}$).

Çıkarım: daha çok tablo = daha çok güç **değildir**.

2015 · DFA (hata enjeksiyonu)

- **DFA (Differential Fault Analysis):** çalışırken bir değere **hata enjekte edip** bozulan çıktıdan anahtarı geri hesaplar.
- "Unboxing the White-Box" (Black Hat EU).

Çıkarım: tasarımın iç matematiğini **bilmeden** de kırılır.

2016 · DCA (en önemlisi)

- **DCA (Differential Computation Analysis):** çalışırken üretilen ara değerlerin **izini** toplar, DPA benzeri istatistikle anahtarı çıkarır.
- İç kodlamaları çoğu zaman **görmezden gelir**.

Çıkarım: klasik yan kanal savunmaları WBC için de gerekli.

DCA saldırısı — şema

DCA: donanım yan kanalının yazılım hâli

güç izi yerine YAZILIM izi kullanılır



Klasik DPA'nın kardeşi. İç kodlama zayıfsa korelasyon kodlamayı aşar.

Sezgi: neyi izliyor?

- WBC çalışırken tablolardan **ara değerler** okur.
- Bu ara değerler, gizli anahtara **bağlı** değişir.
- DCA bu değerlerin izini toplar.

İstatistik nasıl anahtarı verir?

- Saldırgan bir anahtar baytı için **tahmin** yapar.
- Tahmine göre ara değerin nasıl davranması gerektiğini hesaplar.
- Toplanan izlerle **en iyi uyan** tahmin, doğru bayttır.

Bu, DPA'nın (güç analizi) **yazılım** hâlidir.

İç kodlama neden DCA'yı durdurmaz?

- İç kodlama ara değeri **karıştırır** ama **birebir** eşlemedir (bijeksiyon)
- İstatistiksel korelasyon çoğu zaman **hayatta kalır**
- Bu yüzden 2016'dan sonra WBC'ler DCA'ya karşı savunmasız kaldı

2017–2024 · WhibOx yarışmaları

- Akademi ve endüstri tasarımlarını yarışmaya gönderir.
- Gönderilen tasarımların **tamamı kırıldı**.

Çıkarım (bugün): kırılmadan kalan yayımlanmış saf-yazılım WBC **yok**.

Kronoloji · özet

Yıl	Olay	Çıkarım
2002	Chow WB-AES/DES	Doğuş
2004	BGE	İç kodlama yetmez
2007	WB-DES kırıldı	DES bırakılır
2010'lar	Karroumi kırıldı	Çok tablo $\neq$ güç
2015	DFA	Matematiği bilmeden
2016	DCA	Yan kanal savunması gerek
2017–24	WhibOx	Hepsi kırıldı

DCA ve DFA neden yeni tasarımları da vurur?

Çünkü ikisi de tasarımın **gizli matematiğini çözmeye çalışmaz**:

- **DCA**: kutuyu açmaz; çalışırken **iz** toplar, istatistikle anahtarı bulur
- **DFA**: çalışırken **hata enjekte eder**, bozuk çıktıdan hesaplar

Genel ve **taşınabilir** oldukları için her tasarımı tehdit ederler.

Fark/hata karşılaştırması, oyuncak ölçekte (animasyon)

anahtar k = 0x3C (saldırgana GÖRÜNMEZ)

hata maskesi = 0x01 (tek bit çevrilir)

	x=0	x=1	x=2	x=3	x=4	x=5	x=6	x=7	x=8	x=9
y =	31	D8	7F	26	95	3C	E3	8A	F9	A0
yf =	D8	31	26	7F	3C	95	8A	E3	A0	F9
d =	E9	E9	59	59	A9	A9	69	69	59	59

10 girdiden 10'i bilgi taşıyan ($d \neq 0$) bir fark verdi. TEK BAŞINA hiçbir anahtarı vermez; gerçek DFA/DCA bunun gibi YÜZLERCE-BİNLERCE karşılaştırmayı istatistiksel olarak birleştirir (Bölüm 0'daki korelasyon fikri). Bu yüzden bu animasyon kavramsaldir — çalışan bir saldırı aracı DEĞİLDİR.

Fark karşılaştırması — uç durum: hata maskesi sıfır

anahtar k = 0x3C (saldırgana GÖRÜNMEZ)

hata maskesi = 0x00 (tek bit çevrilir)

	x=0	x=1	x=2	x=3	x=4	x=5	x=6	x=7	x=8	x=9
y =	31	D8	7F	26	95	3C	E3	8A	F9	A0
yf =	31	D8	7F	26	95	3C	E3	8A	F9	A0
d =	00	00	00	00	00	00	00	00	00	00

10 girdiden 0'i bilgi taşıyan ($d \neq 0$) bir fark verdi. TEK BAŞINA hiçbir anahtarı vermez; gerçek DFA/DCA bunun gibi YÜZLERCE-BİNLERCE karşılaştırmayı istatistiksel olarak birleştirir (Bölüm 0'daki korelasyon fikri). Bu yüzden bu animasyon kavramsaldir — çalışan bir saldırı aracı DEĞİLDİR.

Peki ne yapacağız? (karşı önlemler)

Araştırma durmadı; ama hiçbir önlem "artık güvenli" demez.

Hepsi **maliyeti artırır** ve **birlikte** kullanılır. Sırayla:

Karşı önlem 1 · Doğrusal olmayan maskeleye

- DCA'nın dayandığı **doğrusal korelasyonu** bozmaya çalışır (ör. Biryukov–Udovenko 2018).
- İstatistiksel saldırıyı zorlaştırır.

Karşı önlem 2 · Dış kodlama (F, G)

- DCA/DFA/BGE'yi zorlaştırır.
- **Ama** gördüğümüz sınır: standart AES uyumluluğunu kırar.

Karşı önlem 3 · Katmanlı savunma

- WBC'nin çevresine 9. haftanın **gizleme** kuralları
- vi. haftanın **RASP**'ı (anti-debug, kurcalama/bütünlük denetimi)
- Amaç: DCA'nın iz toplamasını, DFA'nın hata enjekte etmesini **zorlaştırmak**

Katmanlı WBC — şema

WBC'yi tek başına bırakmayın

her katman saldırıganın kazanç penceresini daraltır



Tek başına WBC = zaman kazandırır; katmanlı WBC = saldırıyı ölçeklenemez kılar.

Karşı önlem 4 · Anahtar yenileme

- Anahtar sık yenilenirse, bir anahtarı çıkarmanın **değeri düşer**
- x. hafta: **kripto-periyot** (bir anahtarın ömrü)

Karşı önlem 5 · Cihaz/sürüm bağlama

- Tablolar yalnız **belirli bir cihazda/sürümde** anlamlı olur
- **Kod sökmeyi** (code lifting) engeller

Karşı önlem 6 · Sunucu tarafı risk denetimi

- Uçtaki koruma ne olursa olsun, işlem **sunucuda** ayrıca değerlendirilir
- Hız sınırı, anomali tespiti, sayaç/ATC denetimi
- **Son ve en güvenilir** savunma

Karşı önlem 7 · Donanıma taşı

- Mümkünse anahtarı **hiç yazılıma koyma**
- TEE, güvenli öge (SE), HSM

Bu, bir sonraki bölümün konusu.

Ana kural (tekrar)

WBC bir **katmandır**, çözüm değil.

Doğru ifade: "anahtar çıkarmayı şu kadar geciktiriyorum; asıl güvencem anahtar yenileme, cihaz bağlama ve sunucu risk denetiminde."

Bölüm 4 — kısa sinama

1. "Hepsi kırıldı" ifadesi kararınızı nasıl etkiler?
2. DCA hangi klasik saldırıya benzer?
3. İç kodlamalar DCA'yı neden her zaman durdurmaz?
4. En güvenilir son savunma hangisi?

Bölüm 4 — cevaplar

1. WBC **tek başına** güvence değil; anahtar yenileme + cihaz bağlama + sunucu denetimi + gizleme ile; mümkünse **donanım**a taşı.
2. **DPA**'ya (güç analizi) — DCA onun yazılım hâlidir.
3. İç kodlama birebir eşlemedir (bijeksiyon); istatistiksel **korelasyon** çoğu zaman hayatta kalır.
4. **Sunucu tarafı** risk denetimi (son ve en güvenilir).

5. WBC'nin katmanlı savunmadaki yeri

Anahtar korumak: seçenekler

WBC tek seçenek değil. Bir anahtarı korumanın yolları var; güç ve maliyet farklı.

Sırayla, en zayıftan en güçlüye.

Seçenek 1 · Sabit dizide düz anahtar

- Anahtar: yazılımda, **açık**
- Güç: **yok**
- Ne zaman: **asla**

Seçenek 2 · Gizlenmiş/parçalanmış anahtar

- Anahtar: yazılımda, gizlenmiş
- Güç: çok düşük
- Ne zaman: yalnız düşük değerli, kısa ömürlü sırlar

Seçenek 3 · WBC + katmanlı savunma

- Anahtar: yazılımda, **tablolar** gömülü
- Güç: orta (geciktirir)
- Ne zaman: donanım kökü yoksa; **anahtar yenileme + sunucu denetimiyle**

Seçenek 4 · TEE / Güvenli Öğe (SE)

- Anahtar: **donanımla yalıtılmış**
- Güç: yüksek
- Ne zaman: cihaz destekliyorsa **tercih edilir**

Seçenek 5 · HSM / SoftHSM

- Anahtar: donanım (ya da yazılım benzetimi) modülünde
- Güç: yüksek (HSM); SoftHSM yalnız test
- Ne zaman: **sunucu tarafı**; anahtar hiç dışarı çıkmaz

Seenekler · tek tablo

Seenek	Anahtar nerede	Gü
Düz dizi	Yazılımda açık	Yok
Gizli/paralı	Yazılımda gizli	ok düřük
WBC + katman	Tablolara gömülü	Orta
TEE / SE	Donanım yalıtık	Yüksek
HSM / SoftHSM	Donanım modülü	Yüksek

Karar kuralı

Anahtarı yazılıma koymak **zorunda değilsen, koyma.**

- Donanım kökü (TEE/SE/HSM) varsa **onu** kullan
- WBC, donanım kökünün **olmadığı** durumda bir **ödünleşim** çözümüdür
- Her zaman: anahtar yenileme + cihaz bağlama + sunucu denetimiyle birlikte

WBC katmanlı savunmada nereye oturur? (animasyon)

katman

etkin katmanlar (özet)

Static TLS key in a fixed array (bad example) -> (yok)

Session-token encryption key (minutes) -> OBF

One-time-password shared secret (30 s) -> OBF

Push-notification token -> OBF

Local database key, no trusted TEE -> WBC,ROTATE,BIND,AUDIT

Mobile wallet signing key, no trusted TEE -> WBC,ROTATE,BIND,AUDIT

Offline license-check key, no trusted TEE -> WBC,ROTATE,BIND,AUDIT

Local database key, device has a trusted TEE -> HWROOT

Device-pairing key, secure element present -> HWROOT

Payment signing key, server-side HSM -> HWROOT

Cloud KMS master key, server-side -> HWROOT

IoT authentication key, secure element present -> HWROOT

12 varlığın hepsi sınıflandırıldı. wbc kararı HER ZAMAN dört katmanı BİRLİKTE etkinleştirir — WBC hiçbir zaman tek başına yeterli değildir.

Katmanlı savunma — uç durum: hepsi WBC

katman

etkin katmanlar (özet)

Client asset #1, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #2, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #3, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #4, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #5, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #6, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #7, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #8, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #9, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT
Client asset #10, value high, no trusted hardware root -> WBC,ROTATE,BIND,AUDIT

10 varlığın hepsi sınıflandırıldı. wbc kararı HER ZAMAN dört katmanı BİRLİKTE etkinleştirir — WBC hiçbir zaman tek başına yeterli değildir.

"Bu anahtarı nasıl korurum?" — 1

Soru 1: Cihazda TEE/güvenli öge var mı?

- **Evet** → anahtarı oraya koy. **Bitti** (en güçlü).
- **Hayır** → devam.

"Bu anahtarı nasıl korurum?" — 2

Soru 2: Bu sunucu tarafı bir anahtar mı?

- **Evet** → HSM/PKCS#11 (üretimde HSM, testte SoftHSM).
- **Hayır (istemci, donanım yok)** → devam.

"Bu anahtarı nasıl korurum?" — 3

Soru 3: Varlık değeri koruma maliyetini haklı çıkarıyor mu?

- **Düşük** → hafif gizleme + kısa ömür yeter.
- **Yüksek** → WBC + katmanlı savunma + yenileme + cihaz bağlama + sunucu denetimi.

"Bu anahtarı nasıl korurum?" — 4

Her durumda:

- Anahtarı **düz gömme**
- Kararı ve gerekçesini **S8'e yaz**
- Kalan riski açıkça belirt

Karar akışı · tek bakış

Anahtar koruma seçenekleri

yukarıdan aşağı: güç azalır, maliyet de azalır

HSM / donanım

anahtar hiç çıkmaz — en güçlü, en pahalı

TEE / Güvenli öge (SE)

yalıtılmış işlemci bölgesi

WBC + bağlama + yenileme

geciktirir, tek başına değil

Saf yazılım WBC

yayımlananların hepsi kırıldı

Düz gömülü anahtar

koruma DEĞİL

Seçim varlığın değerine göre gerekçelendirilir ve S8'e yazılır.

SoftHSM ve PKCS#11 (sunucu köprüsü)

- **PKCS#11**: anahtar modülleriyle konuşmanın standart arayüzü
- Uygulama anahtarın **değerini görmez**; "şunu imzala/şifrele" der
- **SoftHSM**: aynı arayüzü sunan yazılım benzetimi; gerçek donanım koruması **yok** → geliştirme/test için

WBC uçtaki (istemci) problem; bu sunucudaki anahtar problemine yanıt.

6. Hatalı → saldırı → koruma

1 · Hatalı tasarım

```
/* İstemci uygulaması: veri şifreleme anahtarı gömülü */  
static const uint8_t k[16] = { ... }; /* KÖTÜ */
```

Bir istemci, bir veri şifreleme anahtarını sabit diziye gömer.

2 · Saldırı (kavramsal)

1. `strings` + **entropi taraması** → yüksek entropili 16 baytlık blok
2. Hata ayıklayıcı → bu bloğun şifreleme çağrısında kullanıldığını doğrular
3. Anahtar **dakikalar** içinde çıkar

3 · Koruma (katmanlı) — a

- Anahtarı mümkünse **TEE/SE**'ye taşı
- Değilse **WBC tablolarına** göm (düz bayt dizisi kalmasın)

3 · Koruma (katmanlı) — b

- WBC'yi 9. haftanın **gizlemesiyle** ve 6. haftanın **RASP**'ıyla sar
 - DCA için iz toplamak, DFA için hata enjekte etmek zorlaşır
- **Cihaz/sürüm bağla** → kod sökme işe yaramaz

3 · Koruma (katmanlı) — c

- Anahtarı **sık yenile** (kısa kript-periyot)
- **Sunucuda risk denetimi** uygula (hız sınırı, anomali)

4 · Dürüst kalan risk

- Yeterince kararlı bir saldırgan **tek bir cihazda** anahtarı yine çıkarabilir
- Ama artık:
 - Bir kopyayı kırmak **bütün sistemi açmaz** (cihaz bağlama)
 - Çıkarılan anahtarın **ömrü kısa** (yenileme)
 - Sunucu, anormal kullanımı yakalar

Senaryodan çıkan kurallar

- Anahtarı düz gömme
- Donanım kökü varsa kullan
- WBC'yi **asla** tek başına bırakma
- Cihaz bağlama + yenileme + sunucu denetimi

Vaka · kurgu

Bir mobil uygulama, sunucudan indirdiği **oturum anahtarıyla** yerel veriyi şifreliyor.

- Cihazların bir kısmında TEE var, bir kısmında yok.
- Anahtar günde bir yenileniyor.

Nasıl koruruz?

Vaka · TEE olan cihazlar

- Anahtarı **TEE'ye** koy.
- Uygulama anahtarı hiç görmez; TEE içinde şifreleme yapılır.
- En güçlü çözüm; ek WBC gereksiz.

Vaka · TEE olmayan cihazlar

- Anahtarı **WBC tablolarına** göm (düz dizi yok).
- ix. hafta gizleme + 6. hafta RASP ile sar.
- **Cihaz bağla:** tablolar başka cihazda çalışmasın.

Vaka · ortak katmanlar

- Anahtar **günde bir yenileniyor** → çıkarılanın ömrü kısa.
- **Sunucu**, anormal kullanımı (çok istek, tuhaf konum) yakalıyor.
- Böylece bir cihaz kırılrsa bile hasar sınırlı.

Vaka · kalan risk (dürüst)

- TEE'siz cihazda kararlı saldırgan anahtarı çıkarabilir.
- Ama: tek cihazla sınırlı + 24 saat ömür + sunucu denetimi.
- Bu ödünleşim **S8'de açıkça yazılır.**

Vaka · çıkarım

Tek bir "sihirli" koruma yok.

Güç, **katmanların birleşiminden** ve **dürüst kalan risk analizinden** geliyor.

Değerlendirici (12. hafta) tam da bunu arıyor.

WBC'yi **anlamak**, onu **doğru yere koymak** demektir.

7. Proje ve kapanış

Proje · S8 (anahtar koruma gerekçesi) — 1

En az bir hassas anahtar için bir koruma kararı ve **gerekçesi** yazın:

1. Anahtar nerede durur, nasıl korunur? (düz değilse: TEE/SE, WBC, gizleme — hangisi, **neden**)

Proje · S8 — 2

2. Koruma yeterli değilse (ör. saf yazılımdaysanız) bunu **açıkça** yazın ve telafi katmanlarını belirtin:

- anahtar yenileme periyodu
- cihaz/sürüm bağlama
- sunucu risk denetimi

Proje · S8 — 3

3. WBC kullanmadıysanız **neden** kullanmadığınızın gerekçesi:

- maliyet
- donanım kökü var
- varlık değeri düşük

Gerekçeli bir "kullanmadık" **tam** bir yanıttır.

Depoda gerçek anahtar olmasın

- Bütün anahtar dosyaları, tablolar ve örnekler **sentetik** olmalı
- Sertifikasyonda depoda gerçek anahtar = en ağır bulgulardan biri
- Projede de öyle sayılır

Terimler sözlüğü (1)

Terim	Anlam
Kara/gri/beyaz kutu	Saldırganın gördüğü
WBC	Beyaz kutuda çalışan kriptoloji
S-box	Baytı değiştiren sabit tablo
Bijeksiyon	Tersi alınabilen birebir eşleme
İç/dış kodlama	Tablo içi / tüm şifreyi saran gizli dönüşüm

Terimler sözlüğü (2)

Terim	Anlam
Kod sökme	Anahtarı çıkarmadan kodu kopyalama
DCA	Çalışma izlerine DPA benzeri istatistik
DFA	Hata enjekte edip anahtarı hesaplama
Kripto-periyot	Bir anahtarın ömrü
TEE/SE/HSM	Donanım tabanlı anahtar koruması

Kendini sinama (1–6)

1. Üç saldırgan modeli; AES kanıtı hangisini varsayar?
2. "Tek turu çalıştır" ve "hata enjekte et" hangi saldırılara zemin?
3. Diziye gömülü anahtar neden koruma değil? (entropi)
4. Kod sökme nedir, hangi önlem kapatır?
5. $T[x] = S - \text{box}[x^k]$ tek başına neden güvensiz?
6. İç ve dış kodlama farkı; dış kodlamanın sınırı?

Kendini sınaama — cevaplar (1–6)

1. **Kara kutu** (yalnız G/Ç), **gri kutu** (+yan kanal: güç/zaman), **beyaz kutu** (tam erişim). Klasik AES kanıtı **kara kutuyu** varsayar.
2. "Tek turu çalıştır/ara değeri gözle" → **DCA**; "hata enjekte et" → **DFA**.
3. Anahtar **yüksek entropili** bir blok olarak durur → entropi taraması/ `strings` /desenle bulunur (demo `--scan`). Konum değiştirmek gizleme değil.
4. **Code lifting**: tabloyu/rutini olduğu gibi kopyalayıp başka yerde kullanmak. Kapatan: **dış kodlama** (+ cihaz bağlama/sunucu denetimi).
5. `T[x]=S-box[x^k]` doğrudan k 'ye bağlı; tablo S-box ile karşılaştırılıp **k geri hesaplanır**. **İç kodlamayla** maskeleye.
6. **İç kodlama** ara değerleri gizli bijeksiyonlarla maskeler; **dış kodlama** giriş/çıkışı dönüştürür (code lifting'i zorlaştırır). Sınır: **çağırının da eşleşmesi** gerekir → standart AES uyumu bozulur.

Kendini sınaama (7–12)

7. Tablo boyutu/hız mertebesi? Neden seçili işlemlere?
8. "Hepsi kırıldı" kararınızı nasıl etkiler?
9. DCA hangi donanım saldırısına benzer?
10. Anahtar koruma seçeneklerini güç/maliyetle sırala
11. WBC kullanıyorsan eklemen gereken 3 katman?
12. Bir anahtar için S8 gerekçesini üç cümleyle yaz

Kendini sınaama — cevaplar (7–12)

7. Tablolar **yüzlerce KB** (std AES anahtarı 16–32 bayt) ve **~50× yavaş** → yalnız **seçili/kritik** işlemlere uygulanır.
8. WBC'yi tek başına anahtar koruması saymayız; **geciktirici katman** olarak yenileme + bağlama + sunucu denetimiyle kullanırız (mümkünse donanım).
9. **DPA'ya** (Differential Power Analysis); DCA güç izinin yerine **yazılım izini** (bellek erişimi/ara değer) koyar.
10. (zayıf→güçlü) düz gömülü anahtar < kodlanmış tablo (saf-yazılım WBC) < WBC+bağlama/yenileme < **TEE/SE** < **HSM/donanım**. Maliyet aynı yönde artar.
11. **Anahtar yenileme · cihaz/uygulama bağlama · sunucu tarafı denetim/iptal** (+ RASP/bütünlük).
12. Örnek: "DEK, beklemedeki veriyi AES-256-GCM ile korur. HSM'de üretilir/saklanır, düz bellekte tutulmaz. Saf-yazılım whitebox'lar kırıldığı ve varlık değeri yüksek olduğu için **donanım** seçildi."

Özet: bu haftanın tek cümlesi

Yayımlanmış saf-yazılım whitebox'ların hepsi kırıldı; WBC anahtar çıkarmayı **geciktiren bir katmandır** ve yenileme, cihaz bağlama, sunucu denetimiyle birlikte anlam kazanır.

Kaynaklar

- **Güvenli programlama teknik kılavuzu** — WBC saldırgan modeli, whitebox AES'in konumu
- Chow, Eisen, Johnson, van Oorschot — WB-AES (SAC 2002), WB-DES (DRM 2002)
- Wyseur — "Hiding Keys in Software" (kısa giriş)
- BGE (SAC 2004); Bos vd. DCA (CHES 2016); "Unboxing the White-Box" (BH EU 2015)
- WhibOx yarışması sonuçları (2017–2024)

Bir sonraki hafta

12. hafta — Sertifikasyon ve sızma testi planlaması

Bir korumanın "ne kadar dayandığını" ölçmek → bağımsız değerlendirme süreci ve raporlama.

- **9. hafta:** gizleme kuralları (kod) · **11. hafta:** whitebox (anahtar) — bugün · **14. hafta:** otomasyon (Tigress)

Ortak kural: koruma **kırılamazlık değil, gecikme**; güç katmanlardan ve ölçümden gelir.